Cloud-Native AI Model Deployment Platform

Vemuri Venkata Naga Sai Sri Harshad (2300031215), Pinnamaneni Rohith Venkata Sai (2300031556),
Ventrapragada Jyothi Swaroop (2300032529), Mullapudi Gowtham Chowdary (2300033079)
Name of the Supervisor: Mr. R. Ranjith Kumar (Emp. ID: 7026)
Department of Computer Science and Engineering (CSE-3), Koneru Lakshmaiah Education Foundation

Introduction

Objective of your work

To build a cloud-native platform that deploys, serves, scales and monitors machine learning models as containerised microservices, replacing manual deployment with an automated, secure and observable workflow.

Origin of our proposal

Deploying ML models through monolithic, manual processes is hard to scale, secure, version and monitor. A review of 16 recent papers on MLOps, Kubernetes-based inference serving, API-gateway security and edge computing points to containerised microservices with CI/CD and autoscaling as the way forward. This project turns that pattern into a working prototype.

Methods

Methods and Materials

Developers upload a model to the Model Registry (FastAPI + PostgreSQL), which stores its metadata and versions. A GitHub Actions pipeline tests, scans, builds Docker images and deploys them to Kubernetes. Client requests enter through the Spring Boot API Gateway, are authenticated with JWT and role-checked (RBAC), then routed to the Inference Service. Kubernetes self-heals and autoscales pods; Prometheus and Grafana collect metrics. A Raspberry Pi 4 acts as an edge client calling the cloud APIs.

Technology stack

- Frontend: React.js, Vite, Tailwind CSS, Recharts
- Backend: Spring Boot (gateway), FastAPI (registry, inference), PostgreSQL
- Cloud-native: Docker, Kubernetes (Deployments, HPA, Ingress)
- DevOps: GitHub Actions, GitHub Container Registry
- Observability: Prometheus, Grafana
- Security: JWT authentication, role-based access control

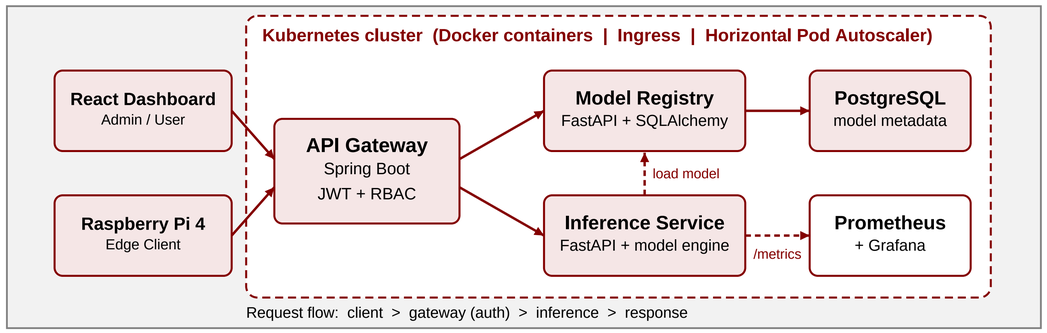
Why these methods

Microservices let each part scale and be updated independently, unlike a monolithic model server. Containers keep deployments portable across AWS, Azure and GCP, Kubernetes adds self-healing and autoscaling, and a gateway centralises security. Hardware is kept low-cost: one Raspberry Pi 4 edge node plus cloud free tier (Rs. 9,000 - 15,000).

Block Diagram, Flowchart, Models, Results

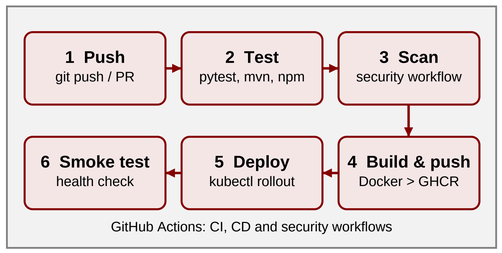
The prototype delivers the complete platform skeleton: three backend microservices, a React dashboard, Docker containers, Kubernetes manifests with autoscaling, CI/CD workflows and a Prometheus/Grafana stack. All 23 automated service tests pass (10 inference, 13 registry). The figures below show the architecture, the delivery pipeline, the monitoring view (simulated data) and the implementation status.

Figure 1



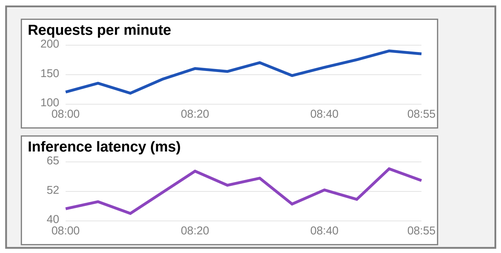
System architecture. Clients (React dashboard, Raspberry Pi 4 edge client) call the Spring Boot API Gateway, which authenticates requests (JWT + RBAC) and routes them to the FastAPI Model Registry and Inference Service running in Kubernetes. Prometheus scrapes service metrics for Grafana.

Figure 2



CI/CD pipeline (GitHub Actions). Every push is tested and scanned; images are built, pushed to GHCR and rolled out to Kubernetes, followed by a smoke test.

Figure 3



Monitoring page of the dashboard: requests per minute and latency. Values are simulated demo data until live Prometheus integration is added.

Figure 4

Component	Technology	Status
API Gateway	Spring Boot, JWT + RBAC	Implemented
Model Registry	FastAPI, SQLAlchemy	Implemented
Inference Service	FastAPI (mock engine)	Implemented
React dashboard	7 pages, mock data	Prototype
CI/CD	3 GitHub Actions workflows	Defined
Kubernetes	24 manifests incl. HPA	Defined
Monitoring	Prometheus + Grafana	Configured
Edge client	Raspberry Pi 4 (in UI)	Simulated

Implementation status of each component at the end of the prototype phase.

Figure 5

Aspect	Value
Gateway HPA	2-8 pods, CPU 70% / Mem 80%
Inference HPA	3-10 pods, CPU 65% / Mem 75%
Registry HPA	2-6 pods, CPU 70% / Mem 80%
Access control	JWT; ADMIN-only delete & upload
Automated tests	23 passed (10 inference, 13 registry)
Edge hardware	Raspberry Pi 4 + 64 GB microSD
Budget	Rs. 9,000 - 15,000 total

Scaling and security settings come from the project manifests; the test count is from running the service test suites.

Conclusion

Discussion/Conclusion

We built a working prototype of a cloud-native platform that covers the model lifecycle: registering, containerising, deploying, serving and monitoring ML models through loosely coupled microservices. Containers, Kubernetes autoscaling and CI/CD remove manual deployment steps, and JWT + RBAC at the gateway secures every request. The design follows the patterns reported in the MLOps and cloud inference-serving literature we reviewed.

Limitations

- The Inference Service uses a mock model engine; real PyTorch, TensorFlow or ONNX models are not loaded yet.
- Dashboard metrics and edge-client data are simulated; Prometheus is not yet connected live.
- Kubernetes manifests and pipelines are defined but not yet validated on a live cloud cluster.
- No load or latency benchmarking has been performed yet.

Future Direction

- Load real model artifacts from the Model Registry into the Inference Service.
- Deploy on an AWS / Azure / GCP free tier and benchmark throughput, latency and autoscaling under load.
- Stream live Prometheus metrics to the dashboard and add alerting.
- Connect the physical Raspberry Pi 4 edge client; add canary rollouts and model-drift monitoring.

References and Affiliations

References

[1] Dominik Kreuzberger, Niklas Kühl, Sebastian Hirschl, "MLOps: Overview, Definition, and Architecture," 2022.
[2] Fmimaw Ayele, Tamirat Abegaz, "On Continuous Integration/Continuous Delivery for Automated Deployment of Machine Learning Models using MLOps," 2022.
[3] "Scalable and Secure AI Inference in Healthcare: A Comparative Benchmarking of FastAPI and Triton Inference Server on Kubernetes," arXiv:2602.00053, 2026.
[4] "Cloud Native System for LLM Inference Serving," arXiv:2507.18007, 2025.
[5] "Scalability Optimization in Cloud-Based AI Inference Services: Strategies for Real-Time Load Balancing and Automated Scaling," arXiv:2504.15296, 2025.
[6] "Secure and Governed API Gateway Architectures for Multi-Cluster Cloud Environments," arXiv:2512.23774, 2025.
[7] "Understanding Practitioners' Perspectives on Monitoring Machine Learning Systems," arXiv:2509.25195, 2025.
[8] "Training Machine Learning Models at the Edge: A Survey," arXiv:2403.02619, 2024.

Acknowledgements

We thank our supervisor, Mr. R. Ranjith Kumar, and the Department of Computer Science and Engineering, KL University, for their guidance and for the lab and cloud resources used in this work.